블랙잭 게임

가상진

프로젝트: 블랙잭 게임 구현

1. Card 클래스

```python
class Card:
    def __init__(self, suit, value):
        self.suit = suit
        self.value = value

    def __str__(self):
        return f'{self.suit} {self.value}'

    def get_value(self):
        if self.value in ['J','Q','K']:
            return 10
        elif self.value == 'A':
            return 11
        else:
            return int(self.value)
```

카드 한 장을 나타내는 클래스이다.

무늬와 숫자를 생성자에 포함하며, J, Q, K는 10점, A는 11점(상황에 따라 1점도 가능)으로 처리된다.

메소드

__str__() 메소드는 카드의 무늬와 숫자를 문자열로 출력한다.

get_value() 메소드는 카드 점수를 반환한다.

2. Deck 클래스

```python
import random

class Deck:
    def __init__(self):
        self.cards = []
        suits = ['Hearts', 'Diamonds', 'Clubs', 'Spades']
        values = ['2','3','4','5','6','7','8','9','10','J','Q','K','A']

        for suit in suits:
            for value in values:
                self.cards.append(Card(suit,value))

    def shuffle(self):
        random.shuffle(self.cards)

    def deal(self):
        if len(self.cards)>0:
            return self.cards.pop()
        else:
            return None

```

전체 카드 덱(52장)을 구성하는 클래스이다.

cards: Card 객체를 담는 리스트

메소드

Shuffle 메소드는 덱을 무작위로 섞는다.

Deal 메소드는 카드 한 장을 덱에서 가져와 반환한다.

이때 cards안에 카드가 있어야 뽑고 한 장도 없으면 None을 반환한다.

3. Hand 클래스

```
38  class Hand:
39      def __init__(self):
40          self.cards = []
41
42      def blackjack(self):
43          return len(self.cards) == 2 and self.getscore() == 21
44
45      def getcard(self, card):
46          self.cards.append(card)
47
48      def getscore(self):
49          score = 0
50          ace = 0
51          for card in self.cards:
52              score += card.get_value()
53              if card.value == 'A':
54                  ace += 1
55          #21점 이상이면
56          while score > 21 and ace > 0:
57              score -= 10
58              ace -= 1
59          return score
60
61
62      def __str__(self):
63          s = ''
64          for card in self.cards:
65              s += str(card) + '  '
66          return s.strip()
67
```

플레이어와 딜러가 가지고 있는 카드와 점수를 관리하는 클래스이다.

cards: 플레이어 또는 딜러가 가지고 있는 카드 리스트

메소드

blackjack메소드는 객체가 가지고 있는 카드가 2장이고 점수가 21점이면 True, 아니면 False를 반환한다.

getcard메소드는 카드 한 장을 얻어서 자기 패에 추가한다.

getscore메소드는 점수를 계산하는 메소드이다.

Card 클래스의 get_value() 메소드를 호출하여 점수에 더한다.

뽑은 카드가 A이면 일단 11점으로 계산하고 ace 변수(A가 몇 개 있는지 세는 변수)를 1 더한다.

그 후 점수가 21점이 넘고, ace가 패에 하나라도 있으면 A는 1점으로 계산되어야 하기 때문에 10점을 빼서 처음부터 1점으로 계산한 것과 동일하게 한다.

__str__메소드는 손에 있는 카드를 문자열로 반환한다.

4. Player 클래스

```
class Player:
    def __init__(self, name='Player'):
        self.name = name
        self.hand = Hand()
        self.chip = 100

    def bust(self):
        return self.hand.getscore() > 21

    def hit(self, deck):
        card = deck.deal()
        if card is not None:
            self.hand.getcard(card)
        return card

    def reset_hand(self):
        self.hand = Hand()

    def __str__(self):
        return self.name + '의 카드: ' + str(self.hand) + '\n점수: ' + str(self.hand.getscore())
```

Player(사용자)를 나타내는 클래스이다.

name: 플레이어 이름

hand: Hand 클래스 객체

chip: 현재 가지고 있는 칩 (기본값 100)

메소드

bust 메소드: 플레이어의 점수가 21점을 초과해서 bust가 됐는지 확인한다.

hit 메소드: Deck 클래스의 deal() 메소드를 호출하여 카드를 한 장 뽑는다.

뽑은 카드가 유효하면 Hand 클래스의 getcard 메소드를 호출하여 패에 카드를 넣고 반환한다.

reset_hand 메소드: 다음 라운드를 하기 위해 패를 초기화하는 메소드이다.

5. Dealer 클래스 (Player 상속)

```
91  class Dealer(Player):
92      def __init__(self):
93          Player.__init__(self, 'Dealer')
94
95      def play(self, deck):
96          print("딜러의 카드:", self.hand)
97          print("딜러 점수:", self.hand.getscore())
98          while True:
99              now = self.hand.getscore()
100             if now >= 17:
101                 break
102             print("딜러가 카드를 뽑습니다:", end=' ')
103             card = self.hit(deck)
104             print(card)
105             print("딜러의 카드:", self.hand)
106             print("딜러 점수:", self.hand.getscore())
107
```

딜러를 나타내는 클래스로, Player 클래스를 상속받아 코드의 중복성을 없앴다.

부모의 생성자를 호출하여 이름을 정한다.

play 메소드: 딜러가 카드를 받고 점수를 받아서 출력한다.

딜러의 점수가 17점 이상일때까지 딜러가 카드를 뽑는다.

6. #Game 클래스 (1) 게임 상태 표시

```python
class Game:
    def __init__(self):
        print("블랙잭 게임에 오신 것을 환영합니다!")
        name = input("이름을 입력하세요: ")
        self.player = Player(name)
        self.dealer = Dealer()

    def start(self):
        while True: #칩이 없을때까지
            if self.player.chip <= 0:
                print("칩이 모두 소진되었습니다. 게임을 종료합니다.")
                break

            print("\n현재 칩:", self.player.chip)
            print("배팅할 금액을 입력하세요: ", end='')
            while True:
                try:
                    coin = int(input())
                    if coin > 0 and coin <= self.player.chip:
                        break
                    else:
                        print("잘못된 금액입니다.")
                except:
                    print("숫자를 입력하세요.")

            from random import shuffle
            self.deck = Deck()
            self.deck.shuffle()
            self.player.reset_hand()
            self.dealer.reset_hand()

            self.player.hit(self.deck)
            self.player.hit(self.deck)
            self.dealer.hit(self.deck)
            self.dealer.hit(self.deck)

            print("\n" + str(self.player))
            print("딜러의 카드:", self.dealer.hand.cards[0], ", Hidden\n")

            if self.player.hand.blackjack():
                print("블랙잭! 이겼습니다.\n")
                self.player.chip += coin
                continue
```

전체 게임의 흐름을 제어하는 메소드이다.

생성자에서 Player의 이름을 받고 Player와 Dealer 객체를 만든다.

start 메소드: 게임을 시작시키고 Player와 Dealer에게 카드 분배, Player 턴과 Dealer 턴을 실행하고, 승패를 결정한다.

Player의 칩이 없을 때까지 반복한다.

만약 Player의 칩이 0 이하이고, 플레이어가 n을 입력하면 종료한다.

현재 칩 개수를 출력하고, 배팅할 금액을 받는다.

이때 예외처리를 사용하여 coin값이 정수 값이 들어오지 않으면 오류가 발생하므로 try-except 블록을 사용해서 예외처리를 하였다.

Deck 객체를 만들어 Deck을 섞고, player와 dealer의 패를 초기화 하고 카드를 각각 2장씩 받는다.

딜러의 2번째 카드는 안보이게 출력하고, player가 블랙잭이 나오면 이기고 칩을 얻는다.

#(2)Game 클래스 플레이어 차례

```
150
151             # 플레이어 턴
152             while True:
153                 try:
154                     move = input("히트(h) 또는 스탠드(s)를 선택하세요: ")
155                     if move == 'h':
156                         self.player.hit(self.deck)
157                         print(self.player)
158                         if self.player.bust():
159                             print("버스트! 졌습니다.\n")
160                             self.player.chip -= coin
161                             break
162                     elif move == 's':
163                         break
164                     else:
165                         raise Exception("h 또는 s만 입력하세요.")
166                 except Exception as e:
167                     print(e)
168
```

플레이어의 점수가 21점이 넘어 버스트가 나거나 s를 입력받을 때 반복문을 멈추게 설계했다.

플레이어가 히트, 스탠드를 선택하게 하고 히트(h)를 선택하면 일단 카드를 한 장 뽑고 그 카드를 출력한다. 버스트인지 확인한 뒤 버스트면 걸은 칩 개수만큼 빼고 반복문을 종료한다.

s를 입력하면 반복문을 종료한다.

입력 받는 중에 h와 s 이외의 입력이 오면 예외처리를 사용하여 raise Exception으로 사

용자에게 메시지를 주어서 다시 입력할 수 있도록 하였다.

#(3) Game 클래스 딜러 차례, 승패 판정 부분

```
168
169             # 딜러 턴
170             if not self.player.bust():
171                 self.dealer.play(self.deck)
172
173                 playerscore = self.player.hand.getscore()
174                 dealerscore = self.dealer.hand.getscore()
175                 print()
176                 print(self.player.name + " 점수:", playerscore)
177                 print("딜러 점수:", dealerscore)
178
179                 if dealerscore > 21:
180                     print("딜러 버스트! 이겼습니다!\n")
181                     self.player.chip += coin
182                 elif playerscore > dealerscore:
183                     print("이겼습니다!\n")
184                     self.player.chip += coin
185                 elif playerscore < dealerscore:
186                     print("졌습니다.\n")
187                     self.player.chip -= coin
188                 else:
189                     print("무승부입니다.\n")
190
191             if self.player.chip <= 0:
192                 print("칩이 모두 소진되었습니다. 게임을 종료합니다.")
193                 break
194
195             while True:
196                 again = input("다시 플레이하시겠습니까? (y/n): ")
197                 if again == 'y':
198                     break
199                 elif again == 'n':
200                     print("게임 종료! 최종 칩:", self.player.chip)
201                     return
202                 else:
203                     print("잘못된 입력입니다. y 또는 n으로 입력하세요.")
204
205 if __name__ == '__main__':
206     game = Game()
207     game.start()
```

딜러의 play() 함수를 호출하여 딜러턴으로 넘어가서 딜러가 카드를 뽑고 점수를 계산하게 한다.

플레이어의 최종 점수와 딜러의 최종 점수를 getscore()함수로 계산한다.

플레이어의 점수와 딜러의 점수를 출력하고, 누가 이겼는지 판단한다.

플레이어가 지고, 칩이 모두 소진되었으면 게임을 끝낸다.

칩이 소진되지 않았다면 다시 플레이를 할지 물어보고 y를 입력하면 while루프를 나와서 다시 게임을 시작하게 하고, n을 입력하면 return 으로 start함수를 종료시킨다.

7.실행결과

```
블랙잭 게임에 오신 것을 환영합니다!
이름을 입력하세요: player

현재 칩: 100
배팅할 금액을 입력하세요: 50

player의 카드: Spades 2  Hearts 4
점수: 6
딜러의 카드: Spades 5 , Hidden

히트(h) 또는 스탠드(s)를 선택하세요: h
player의 카드: Spades 2  Hearts 4  Diamonds 2
점수: 8
히트(h) 또는 스탠드(s)를 선택하세요: h
player의 카드: Spades 2  Hearts 4  Diamonds 2  Hearts 5
점수: 13
히트(h) 또는 스탠드(s)를 선택하세요: h
player의 카드: Spades 2  Hearts 4  Diamonds 2  Hearts 5  Spades K
점수: 23
버스트! 졌습니다.

다시 플레이하시겠습니까? (y/n): y

현재 칩: 50
배팅할 금액을 입력하세요: 50

player의 카드: Diamonds 3  Spades 5
점수: 8
딜러의 카드: Hearts 10 , Hidden

히트(h) 또는 스탠드(s)를 선택하세요: h
player의 카드: Diamonds 3  Spades 5  Hearts 4
점수: 12
히트(h) 또는 스탠드(s)를 선택하세요: h
player의 카드: Diamonds 3  Spades 5  Hearts 4  Diamonds 7
점수: 19
히트(h) 또는 스탠드(s)를 선택하세요: s
딜러의 카드: Hearts 10  Diamonds 8
딜러 점수: 18

player 점수: 19
딜러 점수: 18
이겼습니다!

다시 플레이하시겠습니까? (y/n): n
게임 종료! 최종 칩: 100
```

실행결과가 제대로 실행됨을 볼 수 있다.